

Report A2q1.pdf

Angus Ashby

ID: 33899940

Email: aash0030@student.monash.edu

At a high-level, describe the logical approach you have taken to compute the LCP array using the suffix tree, while reflecting on its worst-case space and time requirement

To construct the LCP array from the suffix tree, I used a depth-first search algorithm approach to traverse nodes in the tree in lexicographical order via sorting of each nodes children, $O(\sigma \log \sigma)$, to create the traversal order.

From this for each child in order, the algorithm searches dfs style until it reaches the deepest leaf, noting the number of characters traversed.

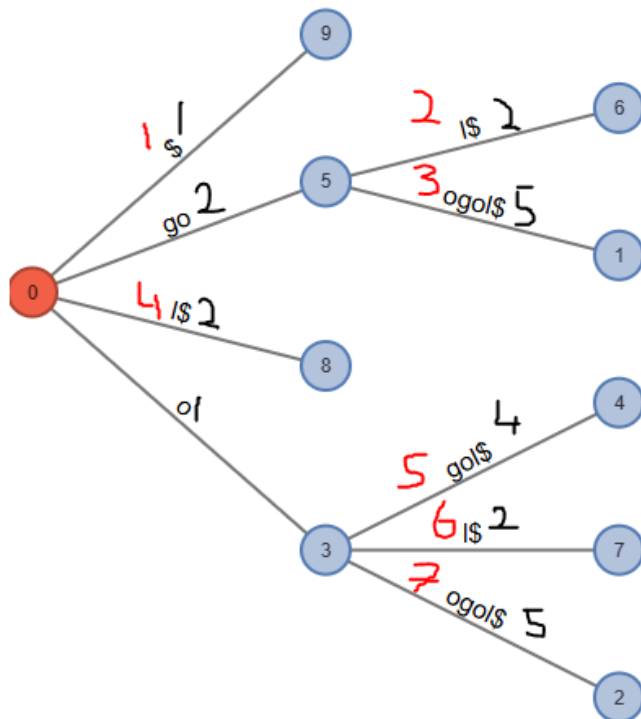
Upon reaching a leaf node, we traverse back up into the next adjacent child again until reaching the bottom.

For each adjacent matching, the 'depth' of the common parent node or Lowest Common Ancestor, is the value of the LCP for that suffix pair, where 'depth' is the number of characters implied in the suffix tree.

This relationship is denoted in the below diagram:

Order = red

length = Black



The worst-case time complexity of this AFTER processing the suffix-tree, is $O(N * \log \sigma)$ and space complexity is $O(n + \sigma)$.

This is because for each node in the tree bounded by N , we sort all children bounded by the size of the alphabet σ .

Further sorting takes $O(d_v * \log d_v)$ to sort a node d 's, d_v children,

and since d_v is $\leq \sigma$, $\log d_v \leq \log \sigma$. The total time spent sorting is bounded as $\log \sigma \cdot \sum d_v$.

Since each d_v counts the children of node v , and each child corresponds to exactly one edge, the sum is bounded by the number of edges E , which is bounded by $O(N)$ via the properties of the suffix tree, so this simplifies to $O(N * \log \sigma)$.

Following the sorting we traverse the suffix tree in usual dfs fashion:

Standard DFS is in $O(V + E)$, and since our suffix tree has exactly N leaves, one per suffix, with at most $N-1$ internal nodes; V is bound by N , and E is bound by $N-1$ so the total is $O(N)$. Giving the final time-complexity $O(N \log \sigma + N) = O(N * \log \sigma)$

Further two lists are stored, one for the LCP bounded by N and one for the sorted children bounded by σ .

The time and space complexity of constructing the suffix-tree is $O(N * \log \sigma)$ via the ukkonen's algorithm with a space complexity of $O(N)$.